

TECHNICAL DOCUMENTATION

TR-Insight

AI-Powered Test Analytics Dashboard — Architecture, Data Flow & Operations Reference

Product: TR-Insight (part of the TR-TestIntelligence suite)

Publisher: Testrig Technologies

Audience: Engineers, DevOps, Solution Architects, Technical Evaluators

Document version: 1.0 | **Date:** September 2, 2026

Table of Contents

1	Executive Summary
2	System Architecture
3	Technology Stack
4	Backend — Module Reference
5	Frontend — Structure & Design
6	Data Ingestion & Storage
7	Authentication, Authorization & Multi-Tenancy
8	LLM Integration Layer
9	AI Chart Generation Pipeline
10	Conversational AI (Chat) Pipeline
11	Live Test Execution Subsystem
12	Deployment & Infrastructure
13	Environment Variable Reference
14	Security Model
15	Testing, QA & Verification
16	API Surface & Access Points
17	Appendix — Glossary

1. Executive Summary

TR-Insight is a self-hosted, full-stack AI analytics platform for software test results. It ingests test output from any existing source — Allure JSON reports, spreadsheets/files, relational databases, generic APIs, or a live Playwright/Cypress test run — normalizes it into a common analytical schema, and exposes it through a role-aware web dashboard, a conversational AI interface, and a deterministic chart-generation engine.

It is the test-reporting member of Testrig Technologies' **TR-TestIntelligence** suite:

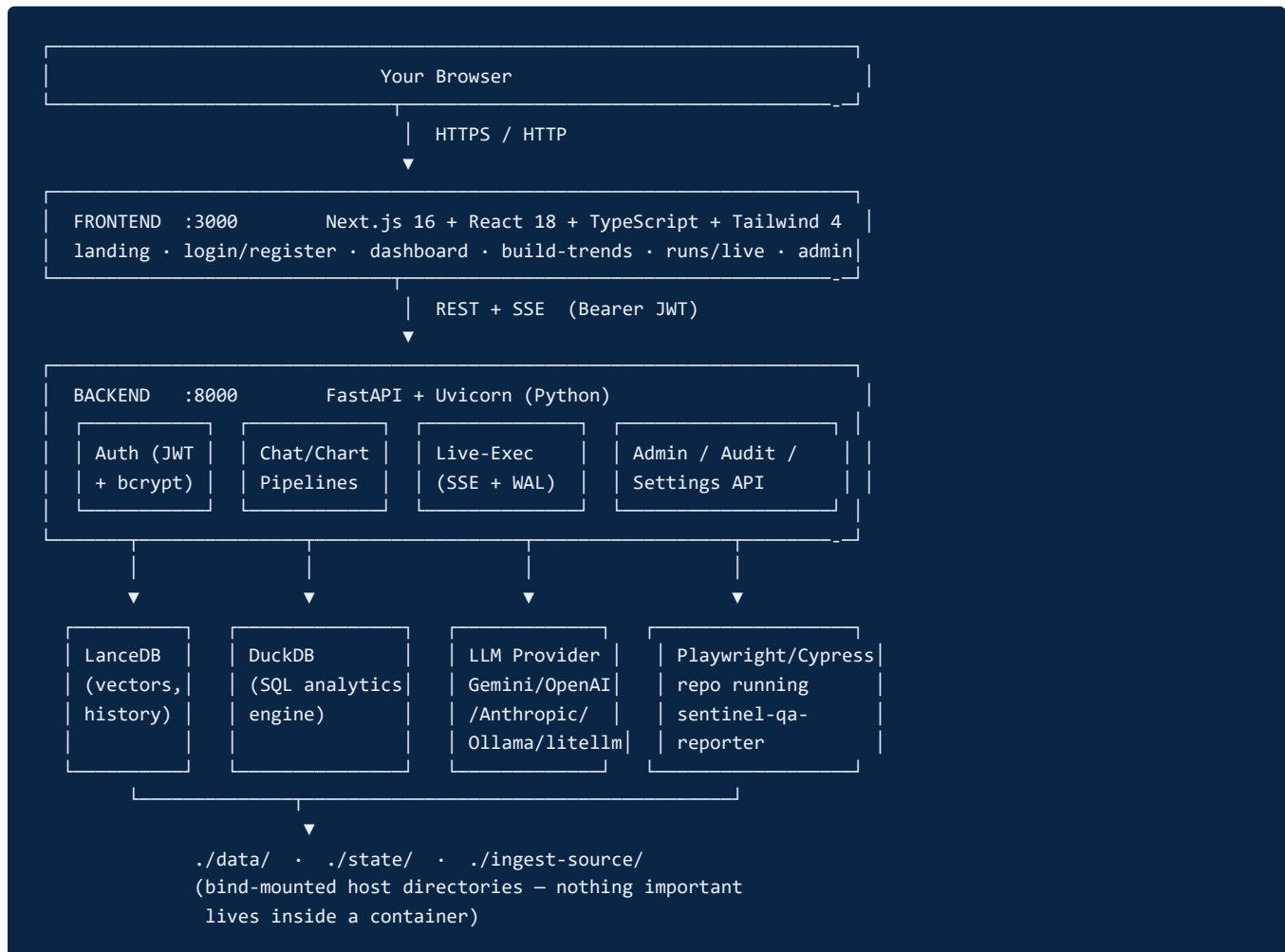
Framework	Purpose
TR-Forge	AI-assisted test case generation
TR-Automate	Playwright test automation with AI assistance
TR-Insight	Test reporting and analytics dashboard (subject of this document)

The architecture separates concerns cleanly: a Next.js frontend, a FastAPI backend, a file-based dual-database storage layer (LanceDB for vector/semantic retrieval, DuckDB for SQL analytics), a pluggable LLM provider abstraction, and an independent live-execution streaming subsystem. There is no external database server or managed service dependency — the entire system runs as one or two containers on a single host, with all state on local disk.

Design principle: the LLM is used only for natural-language understanding and SQL generation — never for producing chart code directly. Chart rendering is deterministic (parsed intent → constrained SQL → validated figure), which makes visual output repeatable, auditable, and immune to LLM hallucination in the rendering path.

2. System Architecture

2.1 High-level component diagram



2.2 Layer responsibilities

Layer	Responsibility
Frontend (<code>frontend/</code>)	All UI — dashboard, chat, chart gallery, build trends, live-run viewer, admin console. Talks to the backend exclusively over HTTP/SSE; no server-side route handlers read the filesystem, so it is deployable independently of the backend host.
Backend (<code>services/</code>)	Auth, permissions, ingestion orchestration, the chat/chart AI pipelines, live-run streaming, the admin API, audit logging, token-quota enforcement.
Universal Ingestor (<code>universal_ingester/</code>)	Connectors (Allure, file, database, API) that normalize any source into the same LanceDB/DuckDB schema.
AI layer	Role/project-aware prompt construction, a deterministic chart-intent parser, and an LLM that only ever writes SQL or plain-English text — never chart-rendering code.

Live-Execution subsystem (<code>services/live_exec/</code> + <code>packages/sentinel-qa-reporter/</code>)	Real-time ingestion of an in-progress Playwright/Cypress run via a small npm reporter package, pushed to the browser over Server-Sent Events.
--	---

3. Technology Stack

Component	Technology
Backend framework	FastAPI, Python, Uvicorn (ASGI)
Authentication	JWT (python-jose), bcrypt password hashing
Frontend framework	Next.js 16, React 18, TypeScript, Tailwind CSS 4
Analytical database	DuckDB (embedded SQL OLAP engine, file-based)
Vector database	LanceDB (embedded vector store, file-based)
Embeddings	Sentence Transformers
AI / LLM	Google Gemini, OpenAI, Anthropic, Groq, Ollama, or any litellm -supported provider
Visualization	Plotly (server-generated figures), Recharts, ApexCharts (frontend components)
Ingestion sources	Allure JSON, flat files (CSV/TSV/Excel/JSON/Parquet/XML/log/PDF/image), MySQL/Postgres, generic REST APIs
Live-run streaming	Server-Sent Events (SSE), SQLite WAL hot layer, in-process pub/sub (Redis for multi-replica)
Containerization	Docker, Docker Compose (split-image and all-in-one variants)
Infra-as-code	Terraform (AWS EC2 reference deployment)

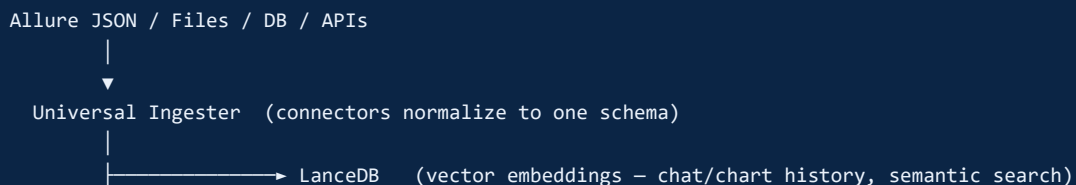
Nothing in this stack requires a managed database service, message broker, or external SaaS dependency for a single-instance deployment — everything is file-based and disk-resident.

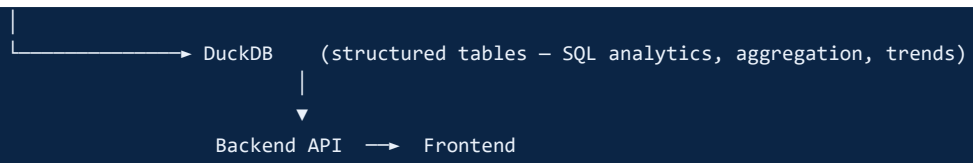
4. Backend — Module Reference

All backend code lives under `services/`, entry point `services/main.py`.

Module	Responsibility
<code>main.py</code>	FastAPI app, CORS configuration, all public and protected routes
<code>auth.py</code>	JWT issuance/verification, bcrypt password hashing, user store (SQLite or in-memory)
<code>permissions.py</code>	Source of truth for role → permission mapping (<code>data.view</code> , <code>data.ingest</code> , <code>data.delete</code> , <code>usage.view_own/view_all</code> , <code>users.manage</code> , <code>settings.manage</code> , <code>audit.view</code>)
<code>data_loader.py</code>	Loads ingested LanceDB/DuckDB tables; abstracts test-result queries
<code>handlers.py</code>	Chat and chart request processing — prompt templates, LLM calls, SQL repair loop
<code>chart_spec.py</code>	Deterministic, regex-based chart-intent parser — no LLM involved: form, measure, dimension, breakdown, top-N, sort direction, filters
<code>chart_builder.py</code>	Deterministic Plotly figure construction from a parsed <code>ChartSpec</code>
<code>chart_theme.py</code>	Validated, colour-blind-safe palette; emits a both-theme (light/dark) payload
<code>llm_client.py</code>	Unified LLM provider abstraction (litellm-backed)
<code>app_settings.py</code>	Runtime LLM settings resolution: env > database > built-in default, decided per field
<code>memory.py</code>	Chat/chart history storage and session management
<code>ingestion_service.py</code>	Orchestrates the Allure → LanceDB/DuckDB pipeline
<code>role_manager.py</code>	Loads AI persona instructions from <code>roles/*.md</code>
<code>project_manager.py</code>	Loads per-project context from <code>projects/*.md</code>
<code>query_executor.py</code>	Executes adaptive SQL against DuckDB with sanitization and error handling
<code>rag_service.py</code>	Semantic retrieval of historical context via vector search
<code>schema_context.py</code>	Generates schema summaries for LLM prompt context, cached per ingestion (capped at 50 ingestions)
<code>audit_log.py</code>	Append-only log of logins, registrations, user/settings edits, credential changes, build deletions
<code>token_usage_store.py</code>	Per-account token usage tracking and quota enforcement
<code>state.py</code>	Global/contextvar-scoped state — the active DuckDB connection and lock per ingestion
<code>live_exec/</code>	The live-execution subsystem — see §11

4.1 Data flow (batch ingestion)





5. Frontend — Structure & Design

Path	Purpose
<code>app/page.tsx</code>	Landing page
<code>app/login/</code> , <code>app/register/</code>	Authentication UI
<code>app/dashboard/</code>	Main analytics UI — KPIs, chat, chart gallery
<code>app/build-trends/</code>	Multi-build comparison charts
<code>app/runs/live/</code>	Live test-run viewer
<code>app/admin/</code>	Admin console (Overview / Users / Usage / Settings / Audit)

Key components

- `FloatingChat.tsx` — conversational AI interface
- `AIGeneratedChart.tsx` — dynamic chart rendering from backend-produced figure specs
- `BuildTrendCharts.tsx` — multi-build trend analysis
- `IngestionSelector.tsx` , `RoleSelector.tsx` , `ProjectSelector.tsx` — context pickers (which build, which AI persona, which project)
- `lib/api.ts` — Axios client with Bearer-token auth

No server-side `app/api/` route handlers — every page/component calls the backend over HTTP via `NEXT_PUBLIC_API_URL` , which is what makes the frontend independently deployable (e.g., to Vercel) from wherever the backend runs. Brand identity (logo, tagline lockup) is centralized in `lib/brand.ts` and shared component set so branding cannot drift page-to-page. Theming uses Tailwind CSS 4 with a persisted `data-theme` attribute for dark/light mode.

6. Data Ingestion & Storage

6.1 Ingestion pathways

Route	Mechanism
Drop-box watcher	<code>AUTO_INGEST_ENABLED=true</code> ; watches <code>ingest-source/</code> for folders/ <code>.zip</code> (Allure) or supported flat files; appears in ~30s; subdirectory name maps to workspace
HTTP upload	<code>POST /ingest/upload</code> with an <code>x-api-key</code> issued as <code>key:workspace</code> pairs (<code>INGEST_API_KEYS</code>) — for CI runners with no shared filesystem
Dashboard "Add Build"	Manual ingestion of a container-visible path, attributed to the logged-in user/workspace
One-shot CLI/container	<code>docker compose run --rm ingest</code> or <code>python ingester.py</code> against <code>config2.json</code> — no backend required to be running

6.2 Storage layout

Path	Default	Contents
<code>DATA_DIR</code>	<code>./data</code>	Every ingestion (<code>ingestion_*</code> / : LanceDB vectors + DuckDB tables + <code>summary.json</code>), finalized live runs (<code>run_*</code> /), <code>live_runs.db</code> (in-progress only), <code>token_usage_store.json</code>
<code>STATE_DIR</code>	<code>./state</code>	<code>users.db</code> , <code>config2.json</code> , auto-generated secrets (<code>secret_key</code> , <code>bootstrap_admin_password</code> , <code>live_ingest_api_key</code>)
<code>INGEST_SOURCE_DIR</code>	<code>./ingest-source</code>	Drop-box for raw data awaiting ingestion

Everything is a bind mount. No state lives inside a container — `docker compose down` , rebuilds, and image deletion never destroy data. Backup is simply copying `DATA_DIR` and `STATE_DIR` (e.g., nightly `rsync` / `tar` to off-host storage); there is no database server to dump.

6.3 Multi-table schema fallback

A historical schema migration renamed `flattened_tests` to `structured_test_results` . `_get_test_results_table()` auto-detects and queries try the new name first, falling back to the old one — so older ingested datasets keep working without re-ingestion.

6.4 Multi-ingestion support

Each ingestion is a fully independent, timestamped dataset. The frontend fetches the list via `GET /ingestions` ; the `x-ingestion-id` header selects which dataset a given API call queries. DuckDB locking is per-ingestion (`state.duck_lock`), so concurrent queries against different ingestions no longer block each other.

7. Authentication, Authorization & Multi-Tenancy

7.1 Credentials & bootstrap

- **JWT + bcrypt** (`python-jose` for tokens, `bcrypt` for password hashing). User store: SQLite by default (`AUTH_BACKEND=db`), or in-memory.
- **Zero-config first run** — a bootstrap admin (`BOOTSTRAP_ADMIN_USERNAME` , default `admin`) is created only while the user table is empty. If `BOOTSTRAP_ADMIN_PASSWORD` is unset, one is auto-generated into `state/bootstrap_admin_password` and logged once at startup.
- That bootstrap account carries `must_change_password=true` ; a middleware blocks every route except `/auth/me` , `/auth/permissions` , and the account-change endpoints until it is cleared — the default password is a one-time door, never a standing credential.
- **Self-generating secrets** — `SECRET_KEY` (JWT signing) and `LIVE_INGEST_API_KEY` (live-run ingest key) both auto-generate into `state/` on first start if left unset, and are reused thereafter.

7.2 Roles vs. permissions vs. personas — three distinct systems

System	Governs	Values
Account role (<code>permissions.py</code>)	What a logged-in user is allowed to <i>do</i>	<code>admin</code> , <code>cto</code> , <code>qa-manager</code> , <code>sdet</code>
Workspace	Which builds a user can <i>see</i>	One per team; non-admins see only their own, admins see all
AI persona (<code>x-role</code> header, <code>roles/*.md</code>)	How the AI <i>phrases</i> an answer, independent of what data is visible	e.g. <code>cto.md</code> , <code>qa-manager.md</code> , <code>sdet.md</code>

`admin` / `cto` are wildcard-everything. `qa-manager` and `sdet` get read/ingest access scoped to their own workspace. Enforcement is server-side — on `GET /ingestions` , every endpoint accepting `x-ingestion-id` , and `DELETE /ingestions/{id}` — not merely a client-side filter.

7.3 Account lifecycle

1. **Bootstrap admin** — from `.env` , applied only while the database is empty.
2. **Self-service registration** — `/register` creates a *pending* account with no login and no workspace until an admin approves it.
3. **Admin-created** — created directly on **Users** → **New user**, already active.

7.4 Token quotas & audit

`users.token_limit` (0 = unlimited) is enforced in `main._enforce_token_quota()` before any LLM call, checked against a lifetime, account-wide total in `token_usage_store` . Every login, registration, user/role/settings edit, credential change, and build deletion is written to an append-only audit log (`audit_log.py` , exposed at `GET /admin/audit`).

8. LLM Integration Layer

`llm_client.py` abstracts over any provider [litellm](#) supports — not a hardcoded list. Configuration resolves per field with a strict precedence:

```
env variable > database (Admin → Settings) > built-in default
```

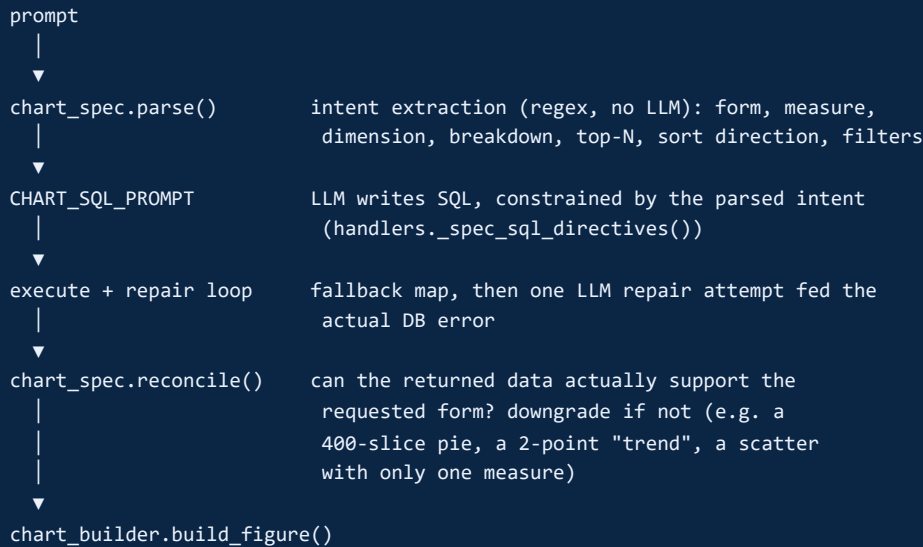
A field pinned in `.env` is locked (shown as "locked by env" in the admin UI) and cannot be overridden live; otherwise an admin can set or change provider/model/key/base-URL at runtime from **Admin** → **Settings**, including a real one-shot connectivity test — no restart required. `LLMClient` re-reads settings on every call rather than caching them at construction time.

Provider	LLM_PROVIDER	Key variable	Example LLM_MODEL
Google Gemini (default)	<code>gemini</code>	<code>GEMINI_API_KEY</code>	<code>gemini/gemini-2.5-flash</code>
Anthropic	<code>anthropic</code>	<code>ANTHROPIC_API_KEY</code>	<code>anthropic/claude-opus-5</code>
OpenAI	<code>openai</code>	<code>OPENAI_API_KEY</code>	<code>openai/gpt-4o-mini</code>
Groq	<code>groq</code>	<code>GROQ_API_KEY</code>	<code>groq/llama-3.3-70b-versatile</code>
Ollama (local, no key)	<code>ollama</code>	—	<code>ollama/llama3</code>

An unconfigured LLM does not block backend startup — chat/chart requests fail clearly with `LLMNotConfiguredError` until a provider is set.

9. AI Chart Generation Pipeline

Charts are built **deterministically**; the LLM's only job in this pipeline is writing SQL.



9.1 Supported chart forms (17)

bar · stacked_bar · horizontal_bar · line · area · pie · donut · scatter · bubble · heatmap · treemap · sunburst · funnel · histogram · box · gauge · radar · `platform_comparison`

9.2 Colour theming

- Colour is assigned **by job, never by rank**: categorical hues in a fixed slot order; a reserved status palette (good/warning/serious/critical) only where colour genuinely encodes state; one hue light→dark for sequential data.
- The palette is validated for colour-blind separation in both light and dark themes (script: `scripts/validate_palette.js`).
- Theming is client-side: the backend emits the light rendering plus both palettes and a per-trace `meta.slot` marker; `frontend/lib/chartTheme.ts` remaps colour by slot on theme change, so a series keeps its visual identity across light/dark.

Every figure carries an **insight subtitle** computed from the drawn data (so it can't drift from what's on screen), a **scope note** when data was truncated or filtered, and a **table-view twin** for accessibility/export. There is deliberately no LLM-generated chart-code execution path — the legacy `CHART_CODE_PROMPT` / `detect_chart_type()` mechanism was removed.

10. Conversational AI (Chat) Pipeline

- `CHAT_DECISION_PROMPT` classifies each question into one of: `sql`, `sql_multi` (a labelled list of queries for multi-part questions), `vector` (semantic/historical retrieval), or `answer` (direct from context).
- SQL errors are fed back to the model for one repair attempt (`handlers._run_sql_with_repair()`) before falling back to a keyword map — the fallback answers a *simpler* question, so it is the last resort, not the first.
- `handlers._extract_json_object()` performs a balanced-brace scan so nested decision objects parse correctly (a bare regex could not).
- Dataset-wide facts (sums, extremes, distributions) are pre-computed over **all** rows and injected into the prompt, since only the first 50 rows are ever sent to the model directly — this keeps aggregate answers accurate without shipping the full dataset to the LLM.
- Requests carry context headers: which ingestion, which AI persona (`x-role`), which project (`x-project`).

11. Live Test Execution Subsystem

Streams a test run to `/runs/live` while it is still executing, then folds it into normal history once finished. Implemented by `services/live_exec/` (server side) and the `sentinel-qa-reporter` npm package (client side).

11.1 Design principles

- **Framework-agnostic by contract** — four plain HTTP calls: `POST /live/runs`, `POST /live/runs/{id}/events`, an optional attachment upload, and `PATCH /live/runs/{id}`. `framework` is free text.
- **One client package, two adapters** — a framework-neutral `src/core/` (HTTP client, CI detection, config resolution, redaction) plus `src/playwright/` and `src/cypress/` adapters exposed as subpath exports. The Cypress adapter splits into a Node half and a browser half bridged over `cy.task`; the browser half stays inert unless the Node half enables it, so an unconfigured repo never invokes the task and reporting cannot break a test run.
- **Never fails a run** — every request has a deadline; the event queue has a ceiling and sheds log lines before test results under pressure; `finish()` is idempotent; the worst possible outcome is a single console warning.
- **withSentinel(config)** (Playwright helper) injects `--remote-debugging-port` into both the top-level and every Chromium project's `launchOptions.args` — required because Playwright *replaces* rather than merges project-level `launchOptions`, so a naive top-level flag would silently never reach the browser in any repo whose projects define their own args.

11.2 Hot layer, sharding, and finalization

- **Hot layer** — SQLite WAL (`store.py`) holds in-progress runs only; a finalize job promotes a completed run into LanceDB and deletes the hot rows. Abandoned runs are swept by `store.reap_stale_runs`.
- **Sharding** — several runner processes can share a `SENTINEL_RUN_KEY` (sent as `external_id`) to attach to one logical run, summing `total_tests / worker_count` and tracking `active_runners`. `update_run_status` only lets the *last* shard finalize the run; an early shard failure is parked in `pending_status` so viewers keep seeing "running" until the whole run is actually done.
- **Push** — in-process pub/sub (`bus.py`) fanned out over Server-Sent Events. A multi-replica deployment needs `REDIS_URL` for this to work across instances.

11.3 Distribution & key management

- Docker images build the reporter's tarball into a `reporter-builder` stage and serve it unauthenticated at `GET /live/package` — intentional, since it is public client code and `npm install` carries no JWT. A team pulling the image gets a client that matches their server by construction, with no npm registry step.
- `GET /live/connection-info` (admin-only) returns this install's real ingest key, a request-derived base URL, and the exact install command, rendered as copy-paste snippets under **Live Runs** → **Connect a repo**.
- `POST /live/connection-info/rotate` reissues the key without a restart, and returns 409 if the key is pinned via environment variable.

12. Deployment & Infrastructure

12.1 Distribution shapes

	Split images (<code>docker-compose.yml</code>)	All-in-one (<code>Dockerfile.allinone</code>)
Containers	2 (<code>backend</code> , <code>frontend</code>)	1 (both processes under <code>supervisord</code>)
Independent restart/scale/limits	Yes	No — both go down together
Best for	A real, operated deployment	A five-minute trial or single-host demo

Pull-only compose files (`docker-compose.pull.yml` , `docker-compose.pull.allinone.yml`) let a team run pre-built images from Docker Hub with no git clone — just the compose file and `.env` .

12.2 Resource sizing

Service	Memory limit	CPU limit
Backend	3 GB	2.0 cores
Frontend	1 GB	1.0 core
Redis (optional, multi-instance only)	256 MB	—

Documented VM starting point: **t3.large** (2 vCPU, 8 GB RAM) — sized for the backend's in-memory embedding model; **t3.medium** is enough to evaluate. Container log files are capped (10 MB × 3) on every service.

12.3 Scaling

- Single instance is the default and needs nothing extra — `WEB_CONCURRENCY=1` by design, since `services/state.py` holds the active ingestion as mutable process state.
- Multiple backend replicas behind a load balancer require `REDIS_URL` (`docker compose --profile multi-instance up`) so the live-run SSE bus and live-frame cache work across instances.
- No database server to scale separately — LanceDB and DuckDB are both file-based under `DATA_DIR` .

12.4 Hosting options

Option	Summary
Docker Compose, any host	Recommended default. Works on a laptop, VM, or bare-metal box.
AWS via Terraform (<code>terraform/</code>)	Provisions one EC2 instance (default t3.large), an Elastic IP, and a security group limited to SSH + 80/443; <code>deploy.sh</code> syncs code and starts it via systemd behind nginx. Approx. cost: t3.large 24/7 ≈ \$60/mo, t3.medium ≈ \$30/mo, 50GB gp3 volume ≈ \$4/mo (US East, on-demand — verify against AWS's current pricing).
Bring-your-own VM	Anything running Docker works. A one-command TLS option exists via a <code>caddy</code> profile (<code>docker compose --profile proxy up -d</code> with <code>PUBLIC_DOMAIN</code> set) — automatic Let's Encrypt, no certbot or nginx config to hand-write.
Split hosting	Frontend on Vercel (built directly from the git repo, no Docker involved), backend on any Docker host with a persistent volume for <code>DATA_DIR</code> / <code>STATE_DIR</code> . <code>NEXT_PUBLIC_API_URL</code> is inlined at Vercel build time —

changing the backend URL needs a redeploy.

Terraform caveat: `terraform destroy` tears down the EC2 instance and its disk. LanceDB/DuckDB data on that instance is *not* backed up automatically — back up `data/` and `state/` before destroying.

13. Environment Variable Reference

Authoritative source: `.env.example` at the repo root (every variable documented inline). Summary by category:

Category	Key variables
Core / bootstrap	<code>SECRET_KEY</code> , <code>LLM_API_KEY</code> , <code>BOOTSTRAP_ADMIN_USERNAME</code> , <code>BOOTSTRAP_ADMIN_PASSWORD</code> — all have safe auto-generated defaults
LLM	<code>LLM_PROVIDER</code> , <code>LLM_MODEL</code> , per-provider keys, <code>LLM_API_BASE</code> , <code>OLLAMA_URL</code>
Storage	<code>DATA_DIR</code> , <code>STATE_DIR</code> , <code>INGEST_SOURCE_DIR</code>
Ports / networking	<code>BACKEND_PORT</code> (8000), <code>FRONTEND_PORT</code> (3000), <code>NEXT_PUBLIC_API_URL</code> (build-time), <code>CORS_ALLOWED_ORIGINS</code>
Ingestion triggers	<code>AUTO_INGEST_ENABLED</code> , <code>AUTO_INGEST_INTERVAL_SECONDS</code> , <code>AUTO_INGEST_STABLE_SECONDS</code> , <code>AUTO_INGEST_DEFAULT_WORKSPACE</code> , <code>INGEST_API_KEYS</code> , <code>INGEST_UPLOAD_MAX_BYTES</code>
Accounts & registration	<code>AUTH_BACKEND</code> , <code>AUTH_AUTO_SEED_USERS</code> , <code>AUTH_SEED_FILE</code> , <code>AUTH_ALLOW_SELF_REGISTRATION</code> , <code>AUTH_AUTO_APPROVE_REGISTRATION</code> , <code>AUTH_DEFAULT_ROLE</code> , <code>AUTH_DEFAULT_WORKSPACE</code> , <code>DEFAULT_WORKSPACE_ID</code> , <code>LEGACY_BUILDS_WORKSPACE</code>
Advanced / tuning	<code>PROJECTS_ROOT</code> , <code>ROLES_ROOT</code> , <code>LIVE_INGEST_API_KEY</code> , <code>INGEST_MAX_FILE_SIZE_BYTES</code> (200MB), <code>INGEST_MAX_ROWS</code> (500k), <code>INGEST_TIMEOUT_SECONDS</code> (600s), <code>INGESTION_POOL_SIZE</code> , <code>MAX_TREND_BUILDS</code> , <code>MAX_CROSS_BUILD_QUERY_BUILDS</code> , <code>WEB_CONCURRENCY</code> , <code>REDIS_URL</code> / <code>REDIS_PORT</code>
Published image (Docker Hub)	<code>IMAGE_NAMESPACE</code> , <code>IMAGE_TAG</code> — ignored by local builds, read only by <code>.pull.yml</code> variants

14. Security Model

- **Self-hosted by design** — the entire system, including all test data and defect detail, runs inside the customer's own environment; nothing is sent to a third-party SaaS backend for storage.
- **Secrets never ship with defaults** — `SECRET_KEY`, `LIVE_INGEST_API_KEY`, and the bootstrap admin password all auto-generate into `state/` and are logged once at startup if not explicitly set; there is no fixed factory credential.
- **Forced credential rotation on first use** — the bootstrap admin account is locked to account-settings-only routes until its password is changed.
- **Password hashing** — bcrypt, configurable cost factor (`BCRYPT_ROUNDS` , default 12, range 10–16); minimum password length enforced (`MIN_PASSWORD_LENGTH` , default 8).
- **Workspace isolation** enforced server-side on every data-bearing endpoint, not just hidden in the UI.
- **Token-spend quotas** checked before every LLM call, closing the "runaway AI bill" risk at the request layer, not just via a dashboard after the fact.
- **Append-only audit trail** of all security-relevant actions, exposed to admins/CTOs via `GET /admin/audit` .
- **CORS allow-listing** (`CORS_ALLOWED_ORIGINS`) — no wildcard origin in a real deployment.
- **Live-ingest key** gates the unauthenticated-by-design `/live/package` download boundary — the download itself is public client code, but reporting a run in requires the per-install key.

Note for security review: because there is no external database server and no data leaves the host, the security review surface for a self-hosted deployment is limited to the host itself, its Docker configuration, TLS termination, and the chosen LLM provider's own data-handling terms (relevant only for the prompt/response traffic actually sent to it — see §8).

15. Testing, QA & Verification

Command	Purpose
<code>python -m pytest tests/ -q</code>	Full backend test suite
<code>python -m pytest tests/test_chart_pipeline.py tests/test_chat_helpers.py -v</code>	Chart pipeline (intent parsing + figure construction) and chat helpers
<code>python -m pytest tests/test_adaptive_ingestion.py -v</code>	Adaptive ingestion tests
<code>python test_edge_cases.py / python test_full_pipeline.py</code>	Integration-level scenario tests
<code>python verify_system.py</code>	Dependency, database, and API sanity check
<code>npm run lint</code> (frontend)	Frontend linting
<code>packages/sentinel-qa-reporter/test/</code>	Reporter-package unit tests covering the "never fail a run" guarantees

16. API Surface & Access Points

Service	URL (local development)
Dashboard	http://localhost:3000
Login	http://localhost:3000/login
Register	http://localhost:3000/register
Build Trends	http://localhost:3000/build-trends
Live runs	http://localhost:3000/runs/live
Admin console	http://localhost:3000/admin
Backend API	http://localhost:8000
Interactive API docs (Swagger)	http://localhost:8000/docs
Health check	http://localhost:8000/health

The full, always-current REST surface (request/response schemas, auth requirements) is generated live by FastAPI at [/docs](#) — it is authoritative over any static description, since it is derived directly from the route definitions.

17. Appendix — Glossary

Term	Meaning
Ingestion	One imported batch of test results, stored as a timestamped folder under <code>data/</code> with its own LanceDB/DuckDB tables
Workspace	A team boundary; every build/ingestion belongs to exactly one
Persona	An AI answer style selected via <code>x-role</code> , independent of account role or data access
ChartSpec	The parsed, structured representation of a chart request, produced deterministically before any SQL is generated
Live run	A Playwright/Cypress test execution being streamed to the dashboard while still in progress
Hot layer	The SQLite WAL store holding an in-progress live run before it is finalized into LanceDB
Finalize	The job that promotes a completed live run from the hot layer into permanent history
litellm	The open-source LLM abstraction library used to support many providers behind one interface